Module 9) Python DB and Framework

- **HTML in Python Theory:**

1) Introduction to embedding HTML within Python using web frameworks like Django or Flask.

Ans:

- Python web frameworks like Django and Flask allow HTML to be used with Python for web development.
- Python handles the backend logic, while HTML is used for the frontend display.
- HTML code is written in template files, not directly inside Python files.
- Python passes data to HTML templates to display dynamic content.
- This separation improves code readability, maintenance, and security.
- Frameworks act as a bridge between user requests, Python logic, and HTML responses.

2) Generating dynamic HTML content using Django templates.

Ans:

- Django uses a template engine to generate dynamic HTML pages.
- Templates are HTML files containing variables and template tags.
- Data is passed from views (Python) to templates using a dictionary.
- {{ }} is used to display dynamic data in HTML.
- {% %} is used for logic like loops and conditions.
- Django templates support loops, conditions, and template inheritance.
- This approach separates business logic from presentation logic.

- **CSS in Python Theory:**

1. **Integrating CSS with Django templates.**

Ans:

- CSS is used to style HTML pages in Django applications.
- Django integrates CSS using static files linked inside HTML templates.
- The {% load static %} tag is used at the top of the template.
- CSS files are placed inside the static folder of the app or project.
- CSS is linked using the {% static 'css/style.css' %} tag.
- This method keeps design (CSS) separate from logic (Python).

2. **How to serve static files (like CSS, JavaScript) in Django.**

Ans:

- Django uses the static files framework to serve CSS and JavaScript files.
- Static files are stored in a folder named static.
- STATIC_URL is defined in settings.py to specify the static file path.
- During development, Django automatically serves static files when DEBUG=True.
- The {% load static %} template tag is used to access static files in HTML.
- This approach allows proper management of CSS, JavaScript, and images.

- **JavaScript with Python Theory:**

1. **Using JavaScript for client-side interactivity in Django templates.**

Ans:

- JavaScript is used to add client-side interactivity to Django web pages.
- It handles actions like form validation, button clicks, and dynamic updates.
- JavaScript runs in the browser, reducing server load.
- It can interact with Django views using AJAX or fetch API.
- Django templates allow embedding JavaScript inside HTML files

2. **Linking external or internal JavaScript files in Django.**

Ans:

- JavaScript files are stored in the static directory of the Django project.
- The {% load static %} tag is used in templates.
- External JavaScript files are linked using <script src="{% static 'js/script.js' %}"></script>.
- Internal JavaScript can be written directly inside <script> tags in HTML.
- This method keeps JavaScript code organized and reusable.

- ## Django Introduction Theory:

1. Overview of Django: Web development framework.

Ans:

- Django is a high-level Python web framework used to build web applications.
- It follows the MVT (Model-View-Template) architecture.
- Django promotes rapid development and clean design.
- It provides built-in features like authentication and admin panel.
- Django is widely used for secure and scalable applications.

2. Advantages of Django (e.g., scalability, security).

Ans:

- Django provides **high security** against common attacks.
- It supports **scalable application development**.
- Built-in admin panel saves development time.
- Django encourages **clean and maintainable code**.
- Large community support and documentation are available.

3. Django vs. Flask comparison: Which to choose and why.

Ans:

- Django is a full-featured framework, while Flask is lightweight.
- Django includes built-in tools like admin and ORM; Flask requires extensions.
- Flask gives more flexibility but needs manual configuration.
- Django is suitable for large and complex projects.
- Flask is better for small or simple applications.

- ## Virtual Environment Theory:

1. Understanding the importance of a virtual environment in Python projects.

Ans:

- Virtual environments isolate project dependencies.
- They prevent conflicts between different Python projects.
- Each project can use a different version of libraries.
- They help maintain a clean system Python installation.
- Virtual environments improve project portability.

2. Using venv or virtualenv to create isolated environments.

Ans:

- venv and virtualenv are tools to create virtual environments.
- venv comes built-in with Python.
- Virtual environments create a separate folder for packages.
- They are activated before installing project dependencies.
- This ensures project-specific dependency management.

- **Project and App Creation Theory:**

1. Steps to create a Django project and individual apps within the project.

Ans:

- Install Django using pip.
- Create a project using django-admin startproject projectname.
- Navigate into the project directory.
- Create an app using python manage.py startapp appname.
- Register the app in INSTALLED_APPS.

2. Understanding the role of manage.py, urls.py, and views.py.

Ans:

- manage.py is used to run and manage the Django project.
- urls.py maps URLs to views.
- views.py contains business logic and handles requests.
- Views return responses such as HTML pages.
- These files work together to handle web requests.

- ## MVT Pattern Architecture Theory:

1. **Django's MVT (Model-View-Template) architecture and how it handles request-response cycles.**

Ans:

- Model handles database structure and data logic.
- View processes user requests and prepares responses.
- Template defines the HTML layout shown to users.
- User request is sent to urls.py.
- View processes the request, fetches data from Model, and sends it to Template.
- Template renders the final response to the browser.

- ## Django Admin Panel Theory:

1. **Introduction to Django's built-in admin panel.**

Ans:

- Django provides a built-in admin panel for managing database records.
- It is automatically available after creating a superuser.
- The admin panel allows CRUD operations on models.
- It is used to manage users, permissions, and application data.
- It saves development time by providing a ready-made interface.

2. **Customizing the Django admin interface to manage database records.**

Ans:

- Models are registered in admin.py to appear in the admin panel.
- Fields can be customized using list_display, search_fields, and list_filter.
- Admin forms can be modified for better data control.

- Custom actions can be added for bulk operations.
- This improves data management efficiency.

- **URL Patterns and Template Integration Theory:**

1. Setting up URL patterns in urls.py for routing requests to views.

Ans:

- urls.py defines URL patterns for the Django project.
- Each URL is mapped to a specific view function.
- The path() function is used to define routes.
- URL patterns help Django decide which view handles a request.
- This enables clean and organized URL routing.

2. Integrating templates with views to render dynamic HTML content.

Ans:

- Views use the render() function to load templates.
- Data is passed from views to templates using context dictionaries.
- Templates display dynamic content using template variables.
- This separates application logic from presentation.
- It makes web pages dynamic and reusable.

- **Form Validation using JavaScript Theory:**

1. Using JavaScript for front-end form validation.

Ans:

- JavaScript validates user input before form submission.
- It checks required fields, formats, and input length.
- Validation improves user experience by showing instant feedback.
- It reduces unnecessary server requests.
- Common validations include email and password checks.

- **Django Database Connectivity (MySQL or SQLite) Theory:**

1. Connecting Django to a database (SQLite or MySQL).

Ans:

- Django uses SQLite as the default database.
- Database settings are configured in settings.py.
- MySQL can be used by installing the required database driver.
- Database credentials are specified in the DATABASES setting.
- Django manages database connections automatically.

2. Using the Django ORM for database queries.

Ans:

- Django ORM allows database operations using Python code.
- It eliminates the need for writing SQL queries.
- ORM supports creating, reading, updating, and deleting records.
- Queries are performed using model classes.
- It improves code security and readability.

- **ORM and QuerySets Theory:**

1. **Understanding Django's ORM and how QuerySets are used to interact with the database.**

Ans:

- ORM stands for Object-Relational Mapping.
- It maps database tables to Python classes.
- QuerySets represent collections of database objects.
- QuerySets are lazy and evaluated only when needed.
- They allow efficient interaction with the database.

- **Django Forms and Authentication Theory:**

1. **Using Django's built-in form handling.**

Ans:

- Django provides form classes to handle user input.
- Forms automatically validate submitted data.
- They reduce repetitive HTML form code.
- Forms integrate easily with models.
- This improves security and code reuse.

2. **Implementing Django's authentication system (sign up, login, logout, password management).**

Ans:

- Django provides a built-in authentication system.
- It supports user sign-up, login, and logout.
- Passwords are securely stored using hashing.
- Authentication views and decorators control access.
- It simplifies user management and security.

- ## CRUD Operations using AJAX Theory:

1. **Using AJAX for making asynchronous requests to the server without reloading the page.**

Ans:

- AJAX allows data exchange without reloading the page.
- It improves application responsiveness.
- JavaScript sends requests to Django views asynchronously.
- Django returns JSON responses.
- AJAX is commonly used for CRUD operations.

- ## Customizing the Django Admin Panel Theory:

1. **Techniques for customizing the Django admin panel.**
    - **Ans:** Customize display using list_display.
- Add filters using list_filter.
- Enable search using search_fields.
- Customize forms and fieldsets.
- Add custom admin actions.

- ## Payment Integration Using Paytm Theory:

1. **Introduction to integrating payment gateways (like Paytm) in Django projects.**

Ans:

- Payment gateways enable online transactions.
- Paytm is a popular payment gateway in India.
- Django integrates Paytm using APIs.
- Transactions are verified using checksum validation.
- This enables secure online payments.

- **GitHub Project Deployment Theory:**

1. **Steps to push a Django project to GitHub.**

Ans:

- Initialize a Git repository.
- Add project files using git add.
- Commit changes using git commit.
- Create a repository on GitHub.
- Push code using git push.

- **Live Project Deployment (PythonAnywhere) Theory:**

1. **Introduction to deploying Django projects to live servers like PythonAnywhere.**

Ans:

- PythonAnywhere is a cloud hosting platform.
- It allows hosting Django applications online.
- Projects are uploaded or cloned from GitHub.
- Virtual environments are configured on the server.
- The application becomes accessible via a public URL.

- **Social Authentication Theory:**

1. **Setting up social login options (Google, Facebook, GitHub) in Django using OAuth2.**

Ans:

- Social authentication allows login via Google, Facebook, or GitHub.
- OAuth2 protocol is used for authentication.
- Django libraries handle OAuth integration.
- Users authenticate without creating new passwords.
- It improves user convenience and security.

• Google Maps API Theory:

1. **Integrating Google Maps API into Django projects.**

Ans:

- Google Maps API is used to display maps in web applications.
- An API key is generated from Google Cloud Console.
- The API is loaded using JavaScript in templates.
- Django passes location data to templates.
- Maps enhance location-based features.